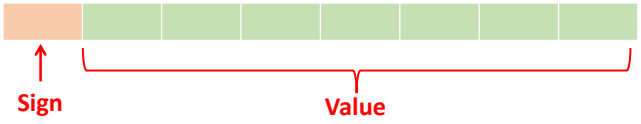
Cx1106 Computer Organisation and Architecture
Computer Arithmetic

Prof Lin WeiSi
School of Computer Science and Engineering, NTU
Email: wslin@ntu.edu.sg

- This chapter is on computer arithmetic.
- You have been introduced many computer arithmetic concepts in Cx1105 and also during the first half of this course so this chapter serves as a supplement of what was taught.

Positive and Negative Numbers

- The content below will focus mainly on the binary system.
- To represent signed integers, computer systems allocate the Most Significant Bit (MSB) to indicate the sign of a number
  - MSB = 0 indicates a positive number
  - MSB = 1 indicates a negative number
- Remaining bits contain the value of the number, which can be interpreted in different ways



- Signed binary integers can be expressed using different number formats, and for this course, we will touch on the most commonly used format in computing
  - Two's Complement Representation

- The rest of the discussion in this chapter will focus mainly on the binary system.
- First, how do we represent signed integers in a binary system?
- In a computer system, the MSB of a binary number is typically used to indicate the sign of a number.
- 1=>negative, 0=> positive.
- There are a few ways in which binary numbers can represented, we will focus on the most commonly used representation system which is the 2's complement number representation system.

Two's Complement Representation

- Positive numbers are represented as normal binary
- Negative numbers are created by negation
  - Invert all bits of the number (flip the bits)
  - Add one to the inverted result (ignoring any overflow)

Example:

+106	**0**	**1**	**1**	**0**	**1**	**0**	**1**	**0**

+106	0	1	1	0	1	0	1	0
Invert	1	0	0	1	0	1	0	1
Add 1	0	0	0	0	0	0	0	1
-106	**1**	**0**	**0**	**1**	**0**	**1**	**1**	**0**

- Most Significant Bit (MSB) indicates the sign of a number
  - MSB = 0 indicates a positive number
  - MSB = 1 indicates a negative number

- For 2's complement representation,, the MSB in a 2's complement number is the signed bit, MSB='1' implies the number is negative and '0' implies the number is positive.
- You can use the negation process to compute the negative equivalent of a positive number and vice versa.
- The negation process involves the following steps
  - First invert all the bits (also known as 1's complement).
  - Then add 1 to the LSB of the result and ignore any bits to the left of the MSB.
- The same process is applied when you want the convert the negative number back to positive.

Two's Complement To Decimal Conversion

Example: What is the decimal representation for 10001110_2

Table of weights

-2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
-128	64	32	16	8	4	2	1
1	0	0	0	1	1	1	0
-128	0	0	0	8	4	2	0

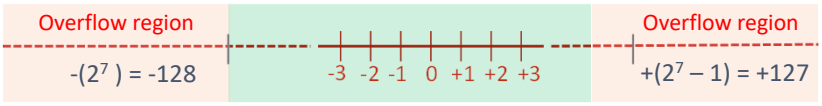
-128 + 8 + 4 + 2 = -114

The decimal representation for 10001110_2 = -114

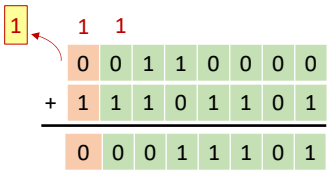
- (Two's Complement → Decimal Conversion, we can use the definition of 2's complement to represent a number: (a positive number → no change); for negative number → invert all bits and +1, as illustrated in the previous slide.)
- Here is the equivalent method that you can use:
  - the most significant bit is not just a sign bit, it also has a negative weightage as shown in the slide.
  - The value of the number is similarly derived by calculating the Sum of Product of the individual bits and their corresponding weightage.
  - Note again that the weightage of the MSB is negative, which is why if the MSB is a '1', the final value will always be negative.

Carry vs Overflow Example

(illustrated with 8-bit representation)



Example: 48 – 19 = 48 + (-19) = 29
First, negate 19 (00010011 -> 11101101)
Then add the two numbers and discard any carries emitting from the high order bit.



- Carry does not always mean that we have an error/overflow
- So how do we detect overflow?

- We will spend the next few slides talking about the difference between the Carry and Overflow Flag in a typical processor.
- Both Carry and Overflow conditional flags were discussed in the first half of the course.
- We understand that Carry Flag is set when there is a ‘1’ that gets carried out of the MSB of the result, as shown in the example in the slide.
- But notice that in this example, 48-19=29 and the results of computation is correct even though the carry bit is set, i.e. a ‘1’ gets carried out of the MSB.
- This means that Carry bit set does not imply that there is an error/overflow in computation for this case, i.e., this is more complex than that of the Carry flag.
    - This case here really applies to the scenario where signed numbers are involved.
    - So Carry bit set in a signed number system doesn’t mean there is an error, and it also doesn’t mean that there is an overflow condition in the result.
- this brings up the next question: how do we detect overflow condition in result?

Detecting Overflow in Two's Complement Numbers

- Overflow can be easily detected by checking the Most Significant Bit (MSB) of the operands and result
- Conditions for overflow
  - In addition (Result = A + B)
    - If MSB(A) = MSB(B), and MSB(Result) ≠ MSB(A)
  - In subtraction (Result = A – B)
    - If MSB(A) ≠ MSB(B) and MSB(Result) ≠ MSB(A)

Operation	Conditions		Result
A + B	A > 0	B > 0	< 0
A + B	A < 0	B < 0	> 0
A – B	A > 0	B < 0	< 0
A – B	A < 0	B > 0	> 0

→ overflow

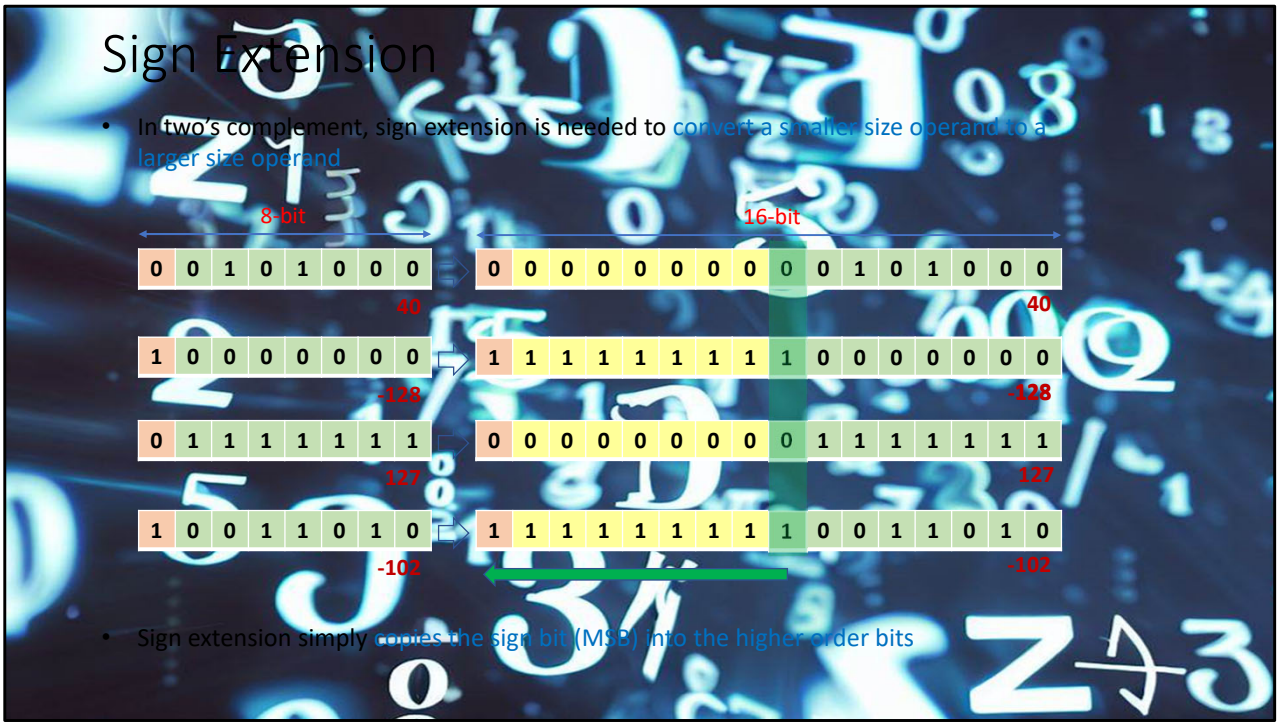
- To detect overflow condition in a signed number system, we need to check the overflow flag.
- Condition required to set the overflow flag is more complex than that of the Carry flag.
  - To detect if the overflow flag needs to be set, the processor needs to evaluate the sign bit of the result and the operands.
- The table illustrates the testing needed to detect overflow when performing addition and subtraction of two numbers A and B.
- Basically, these conditions correspond to the scenario where the results are not feasible. E.g. adding two positive numbers but getting a negative result, adding two negative numbers but getting a positive result.
- Under such condition, the overflow flag is set which implies that there is an error with the result if the system used is a signed number system.

Carry vs. Overflow (recap)

- Unsigned Numbers
  - Carry = 1 always indicates an overflow (new value is too large to be stored in the given number of bits)
  - The overflow flag means nothing in the context of unsigned numbers
- Signed numbers
  - Overflow = 1 indicates an overflow
  - Carry flag can be set for signed numbers, but this does not necessarily mean an overflow has occurred.

Expression	Result	Carry?	Overflow?	Correct Result?
0100 (+4) + 0010 (+2)	0110 (+6)	No	No	Yes
0100 (+4) + 0110 (+6)	1010 (-6)	No	Yes	No
1100 (-4) + 1110 (-2)	1010 (-6)	Yes	No	Yes
1100 (-4) + 1010 (-6)	0110 (+6)	Yes	Yes	No

- A recap on the meaning of carry and overflow.
- In an unsigned number system,
  - Carry Flag set indicates an error as it means that the result is too large to be stored in the given number of bits.
  - While overflow flag has no meaning here.
- For a signed number system,
  - Overflow flag set implies error.
  - Carry flag may be set but it may not imply error has occurred.
- The table below illustrate how the situation described could occur, e.g. In the third row, the Carry flag is set but the results is still correct. -4-2 = -6. Notice that for this case. Overflow Flag is not set.



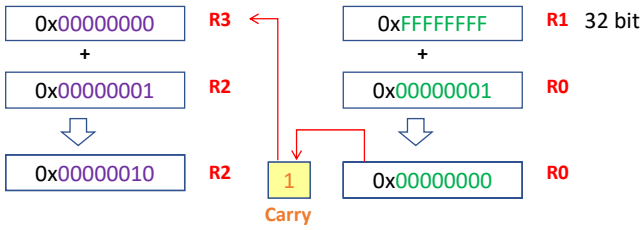
- You have also learnt about sign extension previously
- Its basically the method used when converting a smaller size to a larger size data.
- For example, to convert a number stored in a 8 bit data type to a 16 bit data type variable, sign extension is used to ensure that the values in the two variables are the same.

Multi-Precision Arithmetic

- How can we add operands that are larger than 32-bits (e.g. 64 bit operands) if we have only a single 32-bit ALU?
- The solution is to reuse the 32-bit adder for multi-precision addition
- Multi-precision arithmetic involves the computation of numbers whose precision is larger than what is supported by the maximum size of the processor register (Single-Precision)

```
  00000000 FFFFFFFF
+ 00000001 00000001
  --------------------
  00000010 00000000
  --------------------
```

```
ADD   R0,R0,R1  ; add lower word with carry out
ADC   R2,R2,R3  ; add upper word with carry in
```



- What we have been doing are know as single-precision arithmetic, where operands are kept to the processor data width. For the 32 bit ARM processor, that would mean 32 bit operands.
- So what happen if the operands are larger than 32 bits?
- For example, what do we need to do to add two 64 bits operand?
- Solution is to perform 32bit addition multiple times, and this is illustrated in the slide.
    - The ARM registers are 32bit wide so at any instance, it can only store 32bit data.
    - To add two 64bit operands, we need to add the lower order word first, followed by the higher order word.
    - Note however that when we add the higher order word, the C bit has to be added as well. Carry bit is only set if there are any '1' overflowing from the MSB of the lower order word. This is the same as the carry operation we always do with our paper calculation.
    - ARM assembly instructions wise, the instruction that adds the operands and the carry bit is ADC.

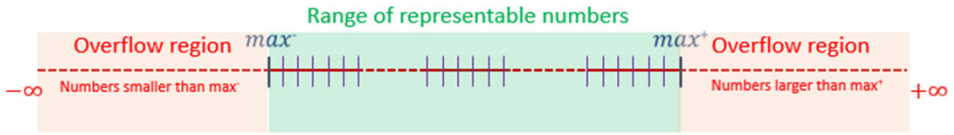
Cx1106 Computer Organisation and Architecture
Fixed and Floating Point Number System

Prof Lin WeiSi
School of Computer Science and Engineering, NTU
Email: wslin@ntu.edu.sg

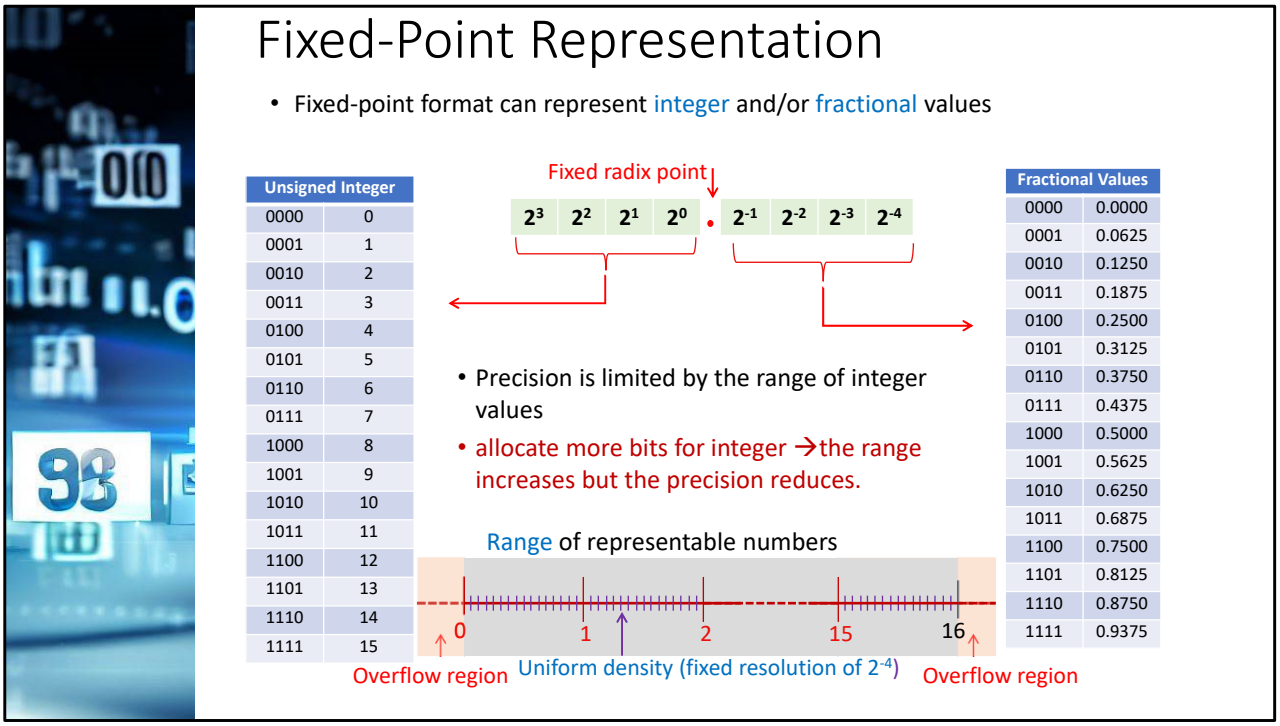
- Next topic in this chapter is the fixed and floating point number system, which is the two main type of number system we typically deal with.

Range and Precision

- Range: Interval between smallest (max^-) and largest (max^+) representable number
  - Example: Range of two's complement is $-(2^{(N-1)})$ to $(2^{(N-1)}-1)$
  - Each tick mark is a representable number in the range
- Precision: Amount of information used to represent each number
  - Example: 1.666 has higher precision than 1.67
  - The number of tick marks provides an indication of precision



- A few definitions before we go into details of this topic.
- Range is the interval between smallest to largest representable number in a system.
- Precision, on the other hand, is the amount of information used to represent each number. E.g. 1.666 represent information in a higher precision compared to 1.67.
- Precision correspond to the interval between adjacent tick marks in the number line shown. Each tick mark correspond to a representable number in the number system used.
- In a binary number system, Precision corresponds to the value represented by the LSB.



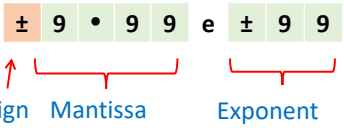
- Let's start with the numbering system which you have been using so far, the fixed point system.
- So far, we have been using fixed point numbers to represent integers.
    - The 4 bit integer you see here has a range of 0 to 15. Any number that is outside this range goes into the overflow region.
    - Similar to the decimal system, binary system has a radix point too and it is known as the binary point.
- And similarly, where digits after the decimal point has a fractional weightage, the bits after the binary points also has a fractional weightage.
- The smallest resolution that this system can support determines its precision, and this as mentioned earlier, correspond to its LSB.
- Fixed point system has a fixed resolution as its radix point position is fixed.
    - For this example, the resolution is 2^-4.
- Some limitation of fixed point system are
    - Given a fixed total number of bits allocated for a number
        - precision is limited by the range of integer.
        - If you allocate more bits for integer, the range will increase but the precision will reduce.
- \

Floating Point Representation

Precision increases for smaller numbers

+999	⇨	$+9.99 \times 10^{+02}$
+99.9	⇨	$+9.99 \times 10^{+01}$
+9.99	⇨	$+9.99 \times 10^{+00}$
+0.999	⇨	$+9.99 \times 10^{-01}$

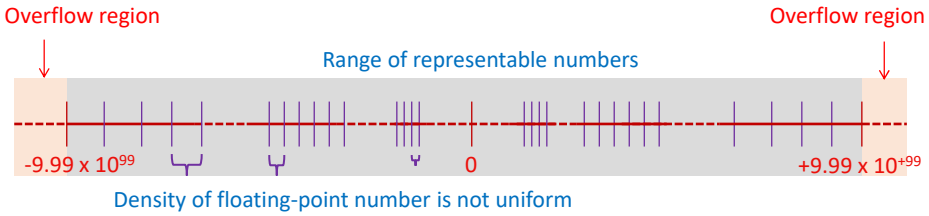
Simple decimal (Radix = 10) floating point format



- Three main fields needed for floating-point representation:
  - Sign – denote positive/negative number
  - Mantissa – base value
  - Exponent – specifies position of radix point

- The slide here shows how we can realise the floating number system that we discussed in the previous slide.
- Take these decimal numbers with different precision for example.
- We can express these numbers in a format which has three fields called sign, mantissa and Exponent.
- By representing number in this manner, we are able to 'move' the position of the radix point by changing the exponent value.
- This is known as floating Point Number system.
- Comparing with a fixed point number, we can see that it doesn't belong to a positional numbering system, actual value is calculated by evaluating the sign, mantissa and exponent value.

Floating Point Representation



- Floating point representation can represent values across a wide range ($-9.99*10^{+99}$ to $+9.99*10^{+99}$).
- Size of exponent determines range of representable numbers.
- Size of mantissa and value of exponent determines the representable precision.
- Small numbers can be represented with good precision. When representing large numbers, precision can be sacrificed to achieve greater range.

- For a floating point number, the size of the exponent determine the range
- While the size of the mantissa, together with the exponent value, determines the precision.
- With this representation format, we are able to have good precision when number value is small, and yet could achieve large range, of course with the trade-off in precision.

Normalisation

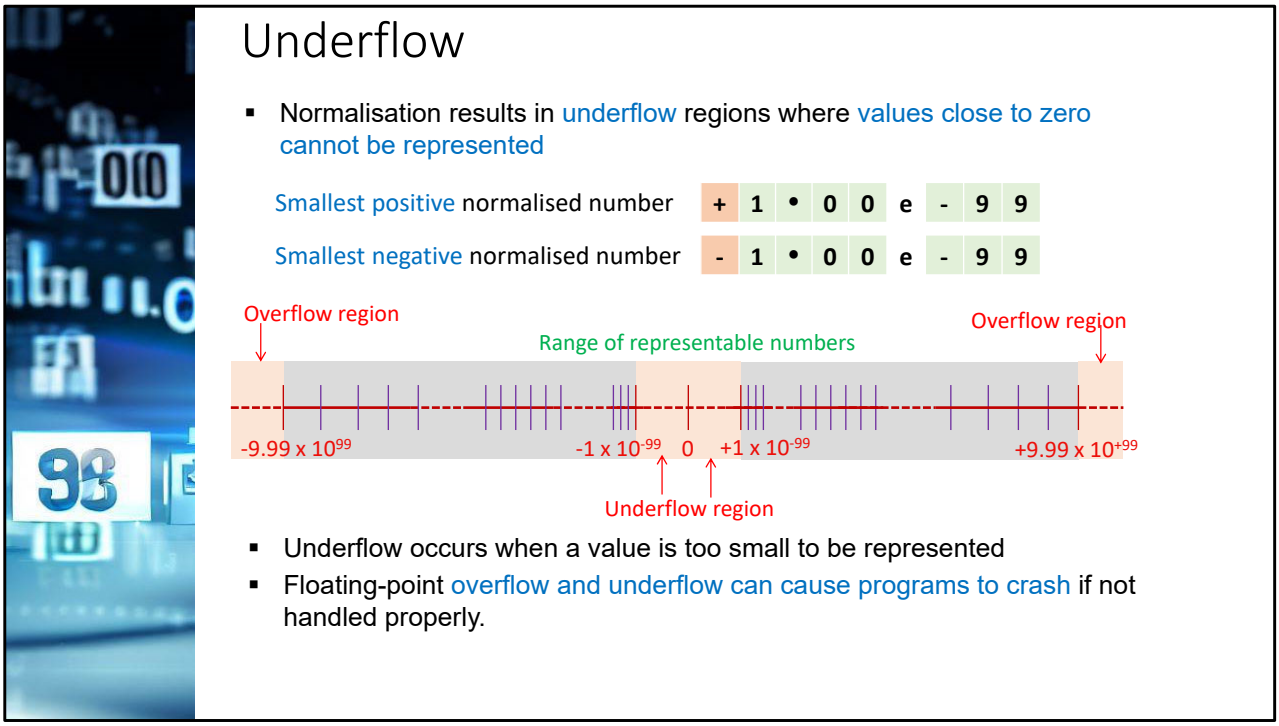
- In the simple decimal floating-point format, there are multiple representations for the same value

±	1	•	0	0	e	±	0	1	**Normalised**
±	0	•	1	0	e	±	0	2	
±	0	•	0	1	e	±	0	3	

- Normalisation is necessary to avoid synonymous representation by maintaining **one non-zero digit before the radix-point**
  - In decimal number, this digit can be from 1 to 9
  - In binary number, this digit should be 1
- Normalisation can maximise number of bits of precision

±	5	•	4	2	e	±	0	3	**Normalised**
±	0	•	5	4	e	±	0	4	

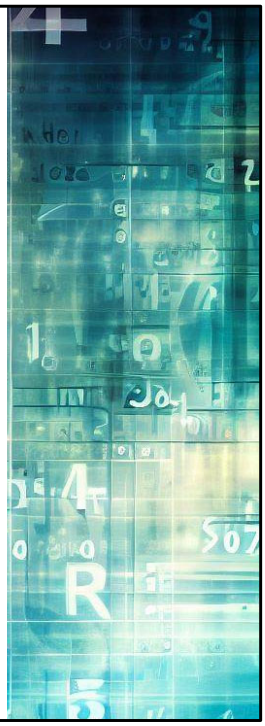
- One issue with floating point numer is that you can have multiple representations for the same number value.
- And this may potentially create confusion.
- So normalization is needed to standardise the representation.
- Normalisaton means that there should be a non-zero digit to the left of the radix point. As shown in the slide.
- Choosing this method of normalization also has an advantage of maximizing the number of bits of representable precision since the number of leading zeroes are minimised.



- But normalization has another side effect of creating underflow region.
- Underflow region refers to region close to zero which cannot be represented by the floating point number.
- In the example here, since the digit to the left of the decimal point has to be non zero for normalised number, that means the smallest positive normalized number is 1*10^-99, very closed to zero but not zero.
- Floating point overflow and underflow may cause the program to crash if not handled properly.
- But you'll see later that typical floating point format standard has some provision to handle the underflow scenario.

Cx1106 Computer Organisation and Architecture
IEEE754

Prof Lin WeiSi
School of Computer Science and Engineering, NTU
Email: wslin@ntu.edu.sg



- This section is on the IEEE754 floating point number standard used in the industry.
- It is also the standard behind the floating point data type you declared in programming languages such as C.

IEEE 754 Floating Point Standard

- Found in virtually every computer since 1980
  - Simplified porting of floating-point numbers
  - Unified the development of floating-point algorithms
- Single Precision Floating-Point Numbers (32-bits)
  - 1-bit sign + 8-bit exponent + 23-bit fraction

- Double Precision Floating-Point Numbers (64-bits)
  - 1-bit sign + 11-bit exponent + 52-bit fraction

- Because of the more complicated format, floating point system needs to be standardised.
- Else program designed by one party may not work with another.
- One of the most commonly used floating point number system is the IEEE754 standard.
- It defined both a single precision and double precision format.
- Single precision format is 32bit wide, has 1 sign bit, 8 Exponent and 23 Fractional bits. This correspond to the Float data type in C.
- Double precision format is 64bit wide, has 1 sign bit, 11 Exponent and 52 Fractional bits. It correspond to the Double data type.
- Note however that the 'Exponent' and 'Fractional' field specify in the IEEE754 format is not the Exponent and Mantissa value of a floating point number. We will touch on the details in the next slide.

IEEE 754 Normalised Numbers

$$(-1)^S \times (1.F)_2 \times 2^{E - Bias}$$

- **Sign** bit
  - S = 0 (positive); S = 1 (negative)
- **Exponent**
  - Biased representation (0000 0001 to 1111 1110)
  - Value of exponent = E - **Bias**
  - **Bias** = **127** (Single Precision) and **1023** (Double Precision)
- **Fraction**
  - Assumes hidden **1.** (not stored) for normalised numbers
  - Value of normalised floating point number is:
    $(-1)^S \times (\mathbf{1} + f_1x2^{-1} + f_2x2^{-2} + f_3x2^{-3} + f_4x2^{-4} + \ldots)_2 \times 2^{E\text{-}Bias}$

- This slide shows how the exponent and fraction field in the IEEE754 number is used to derive the actual value of the floating point number.
- Specifically
  - The Mantissa is derived by attaching a leading '1' to the left of the fractional bits. Giving the expression 1.F.
  - The actual exponent value is given by the expression E-Bias where E is the value from the Exponent Field in a IEEE754 number representation and Bias is = 127 and 1023 for Single and Double precision IEEE754 number respectively.
  - The E field in IEEE754 is a positive number, Bias allows the actual exponent to span across positive and negative number ranges.

Converting Single Precision To Decimal

- Find the decimal value of these single precision number:

0 1 0 1 1 0 0 1 0 1 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

Sign = 0 (positive)

Exponent = 10110010_2 = 178; E – Bias = 178 – 127 = 51

1 + Fraction = $(1.111)_2 = 1 + 2^{-1} + 2^{-2} + 2^{-3} = 1.875$

Value in decimal = $+1.875 \times 2^{51}$

1 0 0 0 0 1 1 0 0 0 1 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

Sign = 1 (negative)

Exponent = 00001100_2 = 12; E – Bias = 12 – 127 = -115

1 + Fraction = $(1.0101)_2 = 1 + 2^{-2} + 2^{-4} = 1.3125$

Value in decimal = -1.3125×2^{-115}

- Some working example for student to check their understanding in deriving the value of a IEEE754 floating point number.
- The first example has a sign bit = 0 so it's a positive number
  - Exponent is 178, so E-127 gives you 51
  - Plug into the expression 1.F*2^(E-Bias) gives the value 1.875*2^51.
- For the second example, sign bit is '1' so the number is negative. Final value is –1.3125*2^(-115).

Representable Range for Normalised Single Precision

- In normalised mode, exponent is from 00000001 to 11111110
- Smallest **magnitude** normalised number

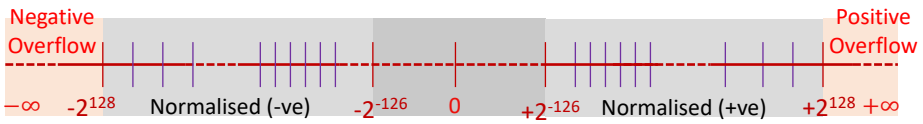
X 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

Exponent = $(00000001)_2$ = 1; E - Bias = 1 – 127 = -126
1 + Fraction = $(1.000…000)_2$ = 1
Value in decimal = **1×2^{-126}**

- Largest **magnitude** normalised number

X 1 1 1 1 1 1 1 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1

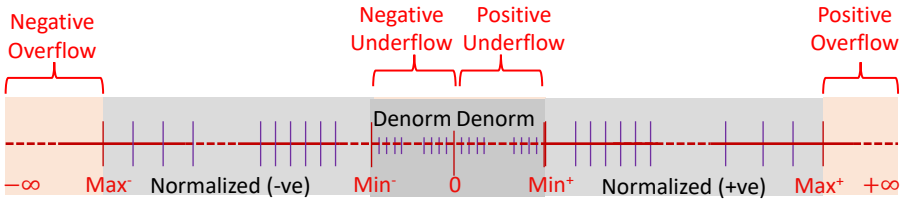
Exponent = $(11111110)_2$ = 254; E - Bias = 254 – 127 = 127
1 + Fraction = $(1.111…111)_2 \approx 2$
Value in decimal ≈ $2 \times 2^{127} = \mathbf{2^{128}}$



- The smallest-magnitude normalised number for single precision IEEE754 standard correspond to E=1 and Fraction=0.
  - That gives you a value of 1*2^(-126)
- The largest-magnitude normalised floating point number correspond to E=11111110 and Fraction=1111…1111 (all ones).
  - That gives a mantissa value of approximately 2 so final value is approximately 2^128.
- We can derive the range for normalised IEEE754 number using the min/max magnitude. As show in the diagram in the slide.
- Note that we didn't use the number 1111 1111 as the largest value of E, neither did we use 0 as its smallest value. Both these numbers are used for special cases in IEEE754. More on that in the next slide.

IEEE 754 Encoding

Mode	Sign	Exponent	Fraction
Normalized	1 / 0	00000001 to 11111110	Anything
Denormalized	1 / 0	00000000	Non zero
Zero	1 / 0	00000000	0000 … 0000
Infinity	1 / 0	11111111	0000 … 0000
Not a Number (NaN)	1 / 0	11111111	Non zero



- As mentioned in the previous slide, some numbers are reserved for special use case in IEEE754.
- The table in this slide illustrate the various use cases.
- In normalised mode, the E value ranges from 1 to 1111 1110 while fraction can take on any value.

- IEEE754 has a special Denormalised mode that allows it to represent numbers in its underflow region.
    - In denorm mode, the MSB of the mantissa is a 0 and not a 1, i.e. 0.F instead of 1.F. Exponent is given a value zero and Fraction can be any non-zero value. Take note however that the exponent value used in the denorm mode is 2^(-126) and not 2^(-127).
    - In any case, this course will not deal into details of denormalised mode.

- Zero, Infinity and NaN are special use cases and is represented by special numbers in E and Fraction.

Fixed Point vs Floating Point Number System

- Given the same number of bits to represent a data, e.g. 32 bits.
- Floating Point (IEEE754)
  - Max Range ≈ $2*2^{128}$ (-2^{128} to ~2^{128}).
  - Max Precision (near to zero) less than 2^{-126}
- Fixed Point
  - Max Range (Radix right of LSB, unsigned) ≈ 2^{32}
  - Max Precision (Radix left of MSB) 2^{-32}
- Floating point yield a larger range and better precision at small numbers with the same number of bits representation.
  - One usually needs the best precision when the numbers are small.
- However, Fixed point number has the advantage of having uniform precision across entire range.
  - Floating point number's precision changes across the range and the very coarse precision at the two end of the range may not be desirable to an intended algorithm.

- Summarising the pros and cons of fixed point vs floating point number representation
- Given the same number of data bits e.g. 32bits
  - The single precision IEEE754 floating point number, when compared to a 32bit fixed point number
    - Has a larger range
    - Is able to achieve a higher precision. This occurs when the number is near to zero.
- So floating point number yield a larger range and better precision compared to a fixed point number occupying the same amount of memory (32 bit for this case).
- However, the fixed point number has the advantage in that the precision value for a fixed point number is uniform across the entire range while that of floating point number varies across the range.
- So depending on the application requirement, you may find one is more suitable over another.